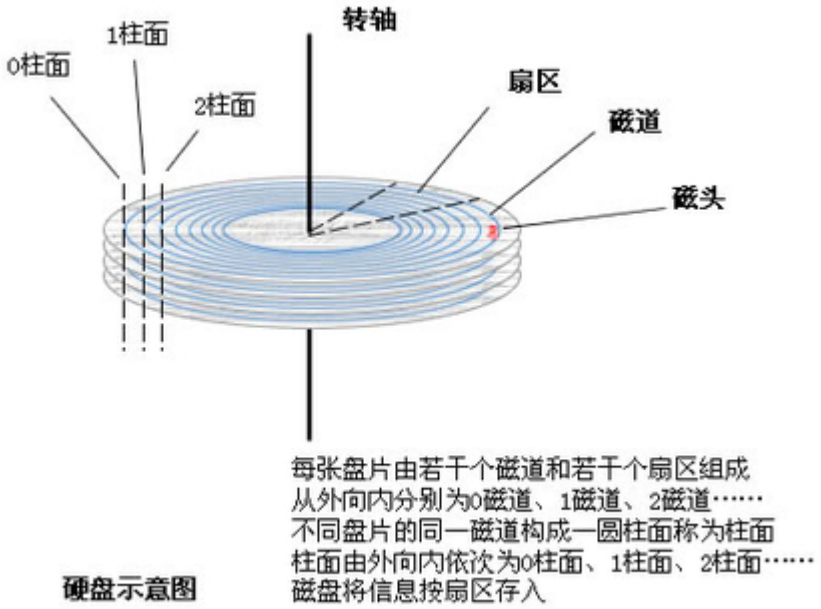
目录

- 什么是磁盘，磁盘的基本结构是什么样的?
  - 操作系统中的一些基本概念:
  - 簇和块有什么区别?
  - 块/簇与页的关系:
- 磁盘的基本存取过程
- 磁盘IO与预读
- 为什么B+树比B树更适合数据库索引?

目录

什么是磁盘，磁盘的基本结构是什么样的?

磁盘示意图如下:



操作系统中的一些基本概念:

- **磁道**:在磁盘上的一圈圈的同心圆。
- **扇区**:磁盘根据角度将每个磁道划分成多个小段，每个小段称为一个扇区，每个扇区就是数据存储的基本单位。每个扇区通常是512字节或4KB，也是磁盘读写的基本单位。
- **块**:块是操作系统获文件系统分配和管理磁盘空间的最小单位，常见大小有512字节、4KB等。，在文件系统、缓存、虚拟内存管理等都以"块"为基本单位。
- **簇**:簇是文件系统（FAT32、NTFS）在块的基础上定义的更大的分配单位。

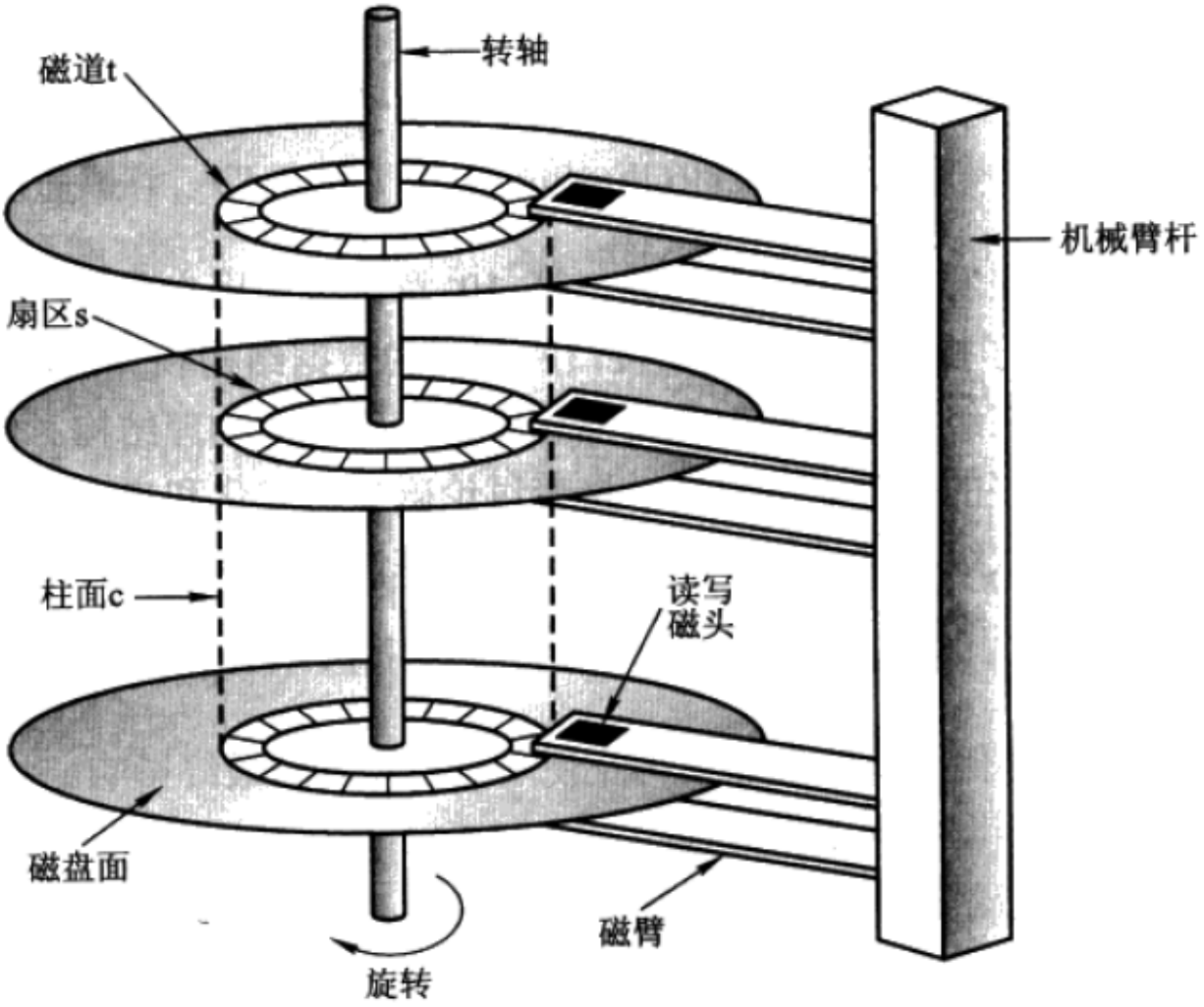
簇和块有什么区别?

通俗的来讲，在Windows下如NTFS等文件系统中叫做簇；在Linux下如Ext4等文件系统中叫做块（block）。每个簇或者块可以包括2、4、8、16、32、64...2的n次方个扇区。

块/簇与页的关系：

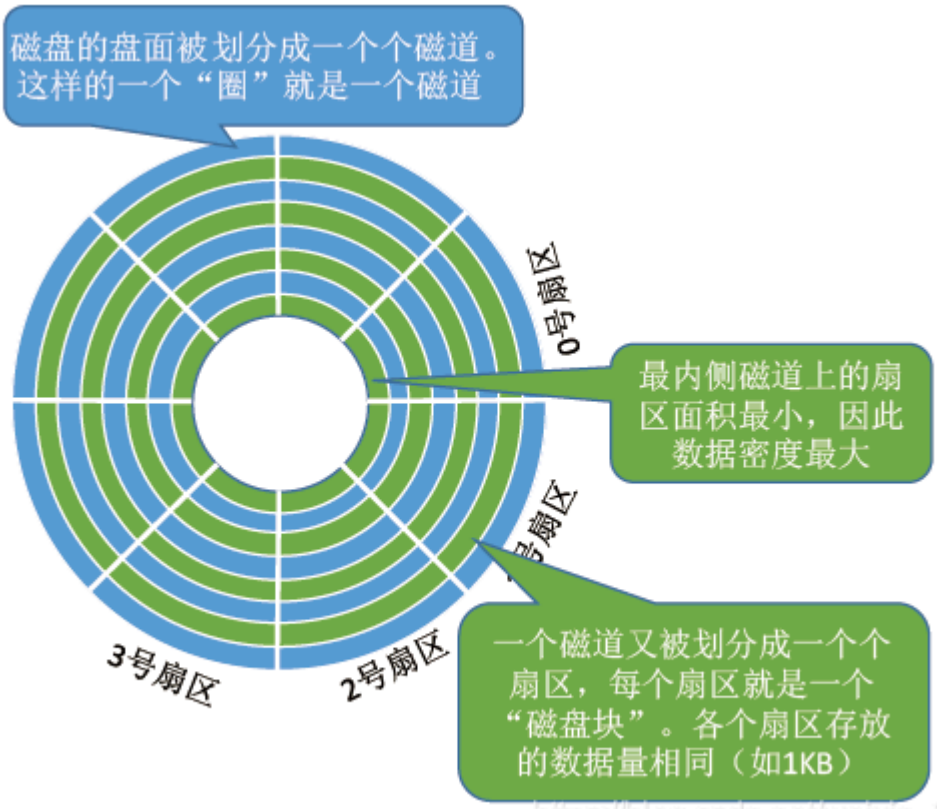
操作系统在与内存和硬盘这两种存储设备进行交互时，都需要定义一种**虚拟的基本单位**来管理和操作数据。

- 与**内存**打交道时，操作系统引入了**“页”的概念**，作为虚拟内存管理的最小单位。
- 与**硬盘**打交道时，操作系统通常以**“块或簇”为最小单位**来进行磁盘 I/O 操作和文件系统管理。



对于一个磁盘。它是一个立体的结构形态，里面会有多个盘面，每个盘面都可以存在一个读写的磁头，这些磁头通过硬件连接在一个主干上面；对于磁盘而言，它是按照一定速度围绕轴心匀速的进行运动的。

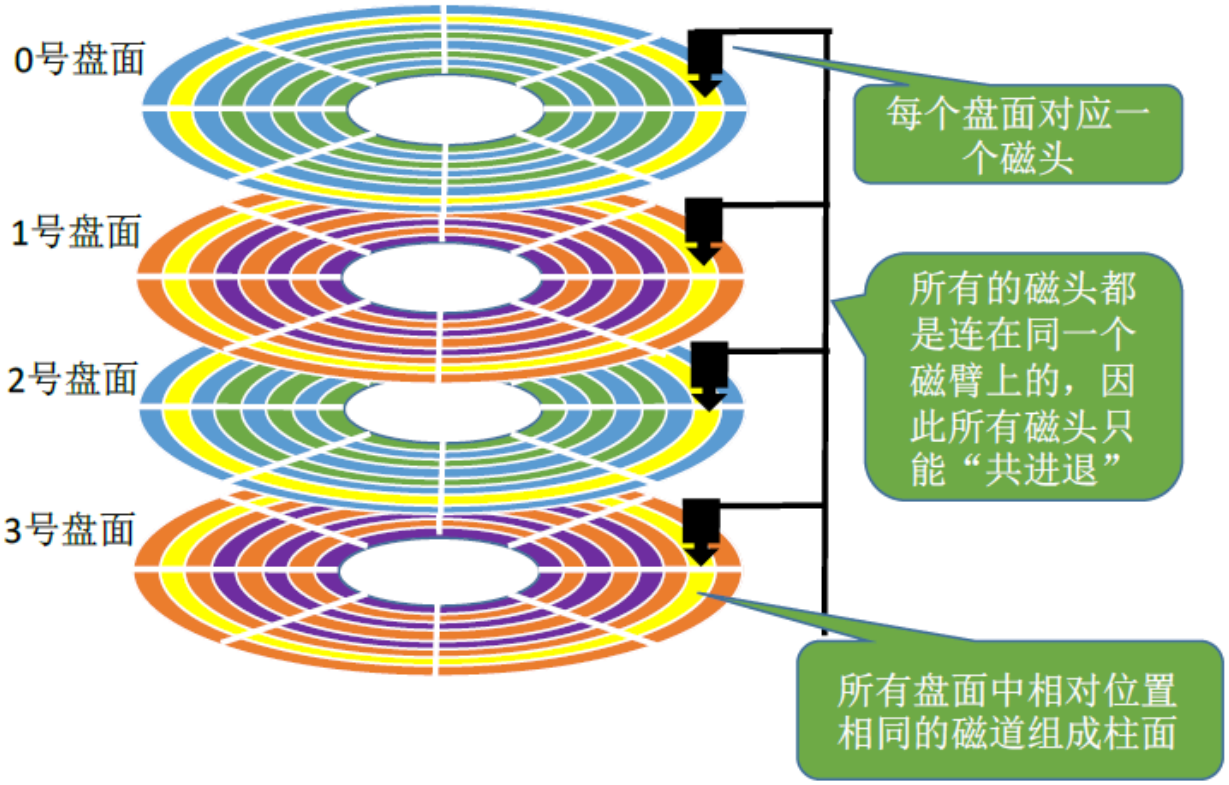
选择一个盘面进行分析：



https://blog.csdn.net/weixin_37641832

对于一个盘面来说，他可以划分为多个同心圆，这个同心圆就叫做磁道，磁道的一圈数据量很大，我们可以根据角度将磁盘进行划分或者说切割，这些分割出来的角度的内容就是扇区，在实际读取的过程中，就是一扇区来读取数据块的，磁道可以有自己的编号，一般是从外往内，最外的磁道为0号磁道，依次为1号磁道，直到N号磁道；

- **柱面：**磁盘是一个立体结构形态，由多个盘片叠加组成，每个盘片有两个盘面。在每个盘面的相同半径处，有一圈磁道，例如所有盘面上的“第0号磁道”。这些位于不同盘面、编号相同的磁道在空间上形成一个柱状结构，称为一个柱面。



磁盘在运行的时候，它查找磁道的过程就是查找柱面的过程。

磁盘的基本存取过程

存取时间 = 寻道时间 + 等待时间

寻道时间：指磁头移动到对应磁道所需要的时间。

等待时间：等待读写的扇区转到磁头下方所用的时间。

我们在读取数据的时候，磁头旋转到对应磁道，等到旋转到相应的扇区的起始位置的时候，就开始读取数据了。

一次IO操作代表什么?

当我们读取磁盘数据的时候应包括一下三个部分：

（1）找磁道的时间。

（2）找扇区的时间。

（3）数据传输时间。

所以我们日常所说的磁盘的IO的成本就是说上面三部分时间的总和。

磁盘IO与预读

计算机存储设备一般分为两种：内存储器(main memory)和外存储器(external memory)。

内存储器为内存，内存存取速度快，但容量小，价格昂贵，而且不能长期保存数据(在不通电情况下数据会消失)。

外存储器即为磁盘读取，磁盘读取数据靠的是机械运动，每次读取数据花费的时间可以分为寻道时间、旋转延迟、传输时间三个部分，寻道时间指的是磁臂移动到指定磁道所需要的时间，主流磁盘一般在5ms以下；旋转延迟就是我们经常听说的磁盘转速，比如一个磁盘7200转，表示每分钟能转7200次，也就是说1秒钟能转120次，旋转延迟就是1/120/2 = 4.17ms；传输时间指的是从磁盘读出或将数据写入磁盘的时间，一般在零点几毫秒，相对于前两个时间可以忽略不计。那么访问一次磁盘的时间，即一次磁盘IO的时间约等于5+4.17 = 9ms左右，听起来还挺不错的，但要知道一台500 -MIPS的机器每秒可以执行5亿条指令，因为指令依靠的是电的性质，换句话说执行一次IO的时间可以执行40万条指令，数据库动辄十万百万乃至千万级数据，每次9毫秒的时间，显然是个灾难。下图是计算机硬件延迟的对比图，供大家参考：

L1 cache reference	0.5 ns
Branch mispredict	5 ns
L2 cache reference	7 ns
Mutex lock/unlock	25 ns
Main memory reference	100 ns
Compress 1K bytes with Zippy	3,000 ns
Send 2K bytes over 1 Gbps network	20,000 ns
Read 1 MB sequentially from memory	250,000 ns
Round trip within same datacenter	500,000 ns
Disk seek	10,000,000 ns
Read 1 MB sequentially from disk	20,000,000 ns
Send packet CA->Netherlands->CA	150,000,000 ns

考虑到磁盘IO是非常高昂的操作，计算机操作系统做了一些优化，当一次IO时，不光把当前磁盘地址的数据，而是把相邻的数据也都读取到内存缓冲区内，因为局部预读性原理告诉我们，当计算机访问一个地址的数据的时候，与其相邻的数据也会很快被访问到。每一次IO读取的数据我们称之为一页(page)。具体一页有多大数据跟操作系统有关，一般为4k或8k，也就是我们读取一页内的数据时候，实际上才发生了一次IO，这个理论对于索引的数据结构设计非常有帮助。

事实1： 不同容量的存储器，访问速度差异悬殊。

- 磁盘(ms级别) << 内存(ns级别)， 100000倍
- 若内存访问需要1s，则一次外存访问需要一天
- 为了避免1次外存访问，宁愿访问内存100次...所以将`最常用`的数据存储在最快的存储器中

事实2： 从磁盘中读 1 B，与读写 1KB 的时间成本几乎一样

从以上数据中可以总结出一个道理，索引查询的数据主要受限于硬盘的I/O速度，查询I/O次数越少，速度越快，所以B树的结构才应需求而生；B树的每个节点的元素可以视为一次I/O读取，树的高度表示最多的I/O次数，在相同数量的总元素个数下，每个节点的元素个数越多，高度越低，查询所需的I/O次数越少；假设，一次硬盘一次I/O数据为8K，索引用int(4字节)类型数据建立，理论上一个节点最多可以为2000个元素，$2000 * 2000 * 2000$ =8000000000，80亿条的数据只需3次I/O（理论值），可想而知，B树做为索引的查询效率有多高；

另外也可以看出同样的总元素个数，查询效率和树的高度密切相关

为什么B+树比B树更适合数据库索引?

1）**B+树的磁盘读写代价更低**

B+树的内部结点并没有指向关键字具体信息的指针。因此其内部结点相对B 树更小。如果把所有同一内部结点的关键字存放在同一盘块中，那么盘块所能容纳的关键字数量也越多。一次性读入内存中的需要查找的关键字也就越多。相对来说IO读写次数也就降低了；

2）**B+树查询效率更加稳定**

由于非终结点并不是最终指向文件内容的结点，而只是叶子结点中关键字的索引。所以任何关键字的查找必须走一条从根结点到叶子结点的路。所有关键字查询的路径长度相同，导致每一个数据的查询效率相当；

3）**B+树便于范围查询（最重要的原因，范围查找是数据库的常态）**

B树在提高了IO性能的同时并没有解决元素遍历的我效率低下的问题，正是为了解决这个问题，B+树应用而生。B+树只需要去遍历叶子节点就可以实现整棵树的遍历。而且在数据库中基于范围的查询是非常频繁的，而B树不支持这样的操作或者说效率太低；不懂可以看看这篇解读-》范围查找

> 参考来源
>
> https://www.cnblogs.com/lianzhilei/p/11250589.html
>
> https://blog.csdn.net/weixin_37641832/article/details/103217311
>
> https://www.cnblogs.com/valor-xh/p/6669897.html
>
> https://blog.csdn.net/weixin_47187147/article/details/126908793